content-engine

GUIDE

The Folder System for AI: A Workspace Blueprint

For anyone who lives in Claude or ChatGPT all day: turn a folder of plain text files into a routed workspace so the AI only ever reads what the task needs.

content-engine (Will Vessels) · 2026-06-27

For: People who use Claude or another AI daily and keep hitting context limits, without needing to be a programmer

Learning objectives

- Explain why dumping everything into one file wastes tokens and breaks down as your work grows.
- Separate your work into focused workspaces instead of one giant context.
- Build the three layer system: a map file, per workspace context files, and the actual folders.
- Write the routing table that tells the AI which files to read and which to skip for each task.
- Adapt the blueprint to your own field by renaming the workspaces and rules.

A folder is the workspace

Most people use AI one chat at a time. They open Claude or ChatGPT, type a long prompt, paste in some notes, get something back, and then start a new conversation when the old one fills up. It works for a while, but it does not scale, and it is not really a workflow.

This guide walks through a different approach: treating a plain folder of text files as your workspace. You point the AI at the folder, and a small set of files tells it where everything is, what to read for the task at hand, and what to ignore. By the end you will understand the whole pattern and be able to build your own version of it.

> **NOTE**
>
> You do not need to be a programmer, and you do not need any special software. Everything here is just folders and plain text files written in normal English.

Why one big file falls apart

AI reads everything you give it and measures it in **tokens**. A token is roughly three quarters of a word, sometimes a whole word. A long word like *hamburger* can be three tokens on its own. The idea comes from older language research that needed a unit smaller than a word, because language does not always break in neat pieces.

The AI can only hold so many tokens at once. That limit is the **context window**, which just means how many tokens the AI can see at one time, and it is finite. When you dump everything into one file, an AI writing a blog post is also reading your video production notes. You are burning the limited window on things that do not matter to the task.

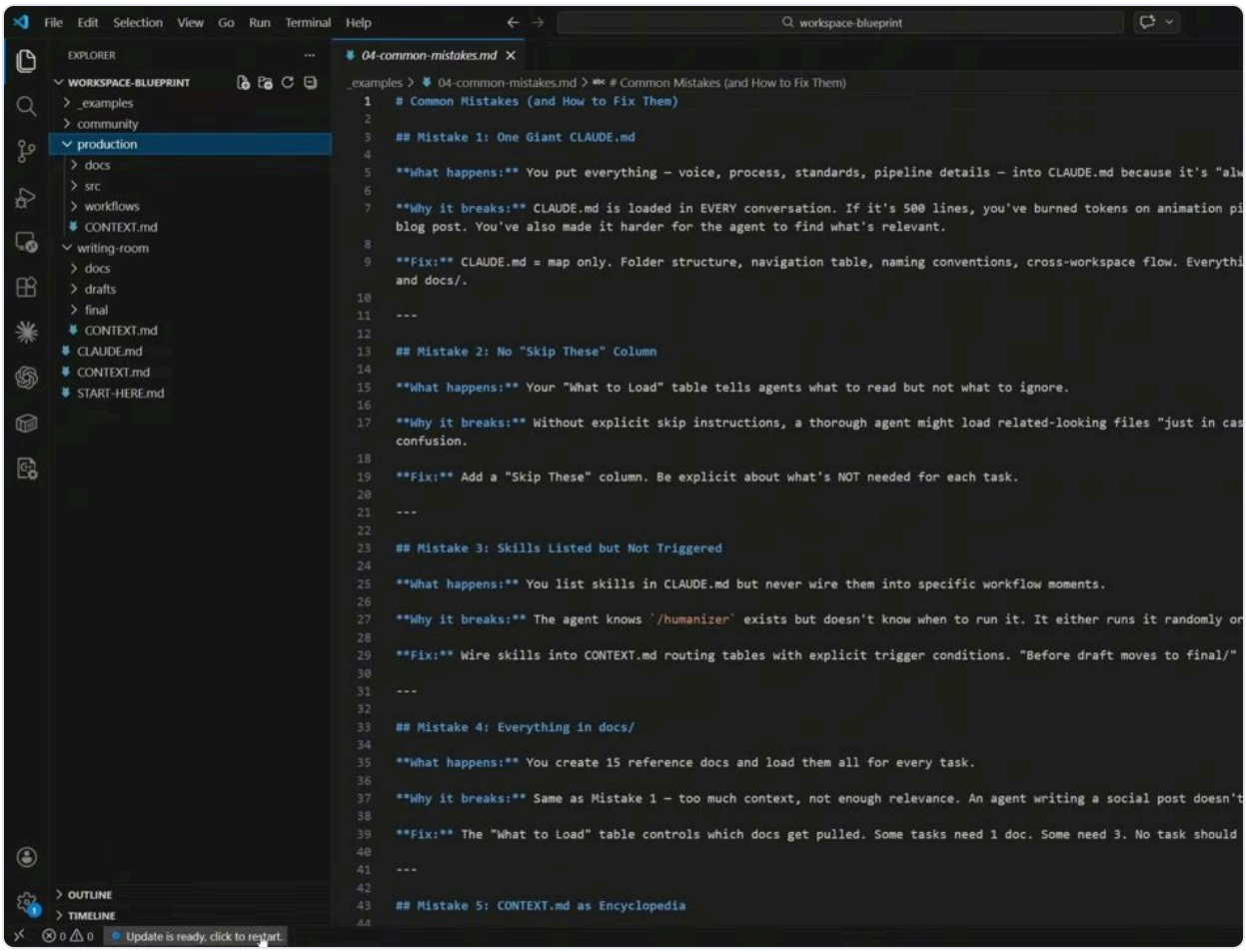
> **WARNING**
>
> Constantly pasting context in, hitting the wall, and starting a fresh conversation is not sustainable. The fix is not a longer prompt. It is keeping unrelated work in separate places so the AI only sees what it needs.

Separate work into workspaces

Instead of one giant file, you separate your thoughts, ideas, and work into different areas. The example used here is a workspace blueprint with three workspaces, and each one handles a

different kind of work. One is a community space for content and docs, one is production for building things like scripts and animations, and one is a writing room for drafting.

Three is just the example. You might have a space for planning, a client list, or anything else your work calls for. The point is that each area does its job well because you can stop the AI from seeing everything at once and instead direct it only to the area you want.

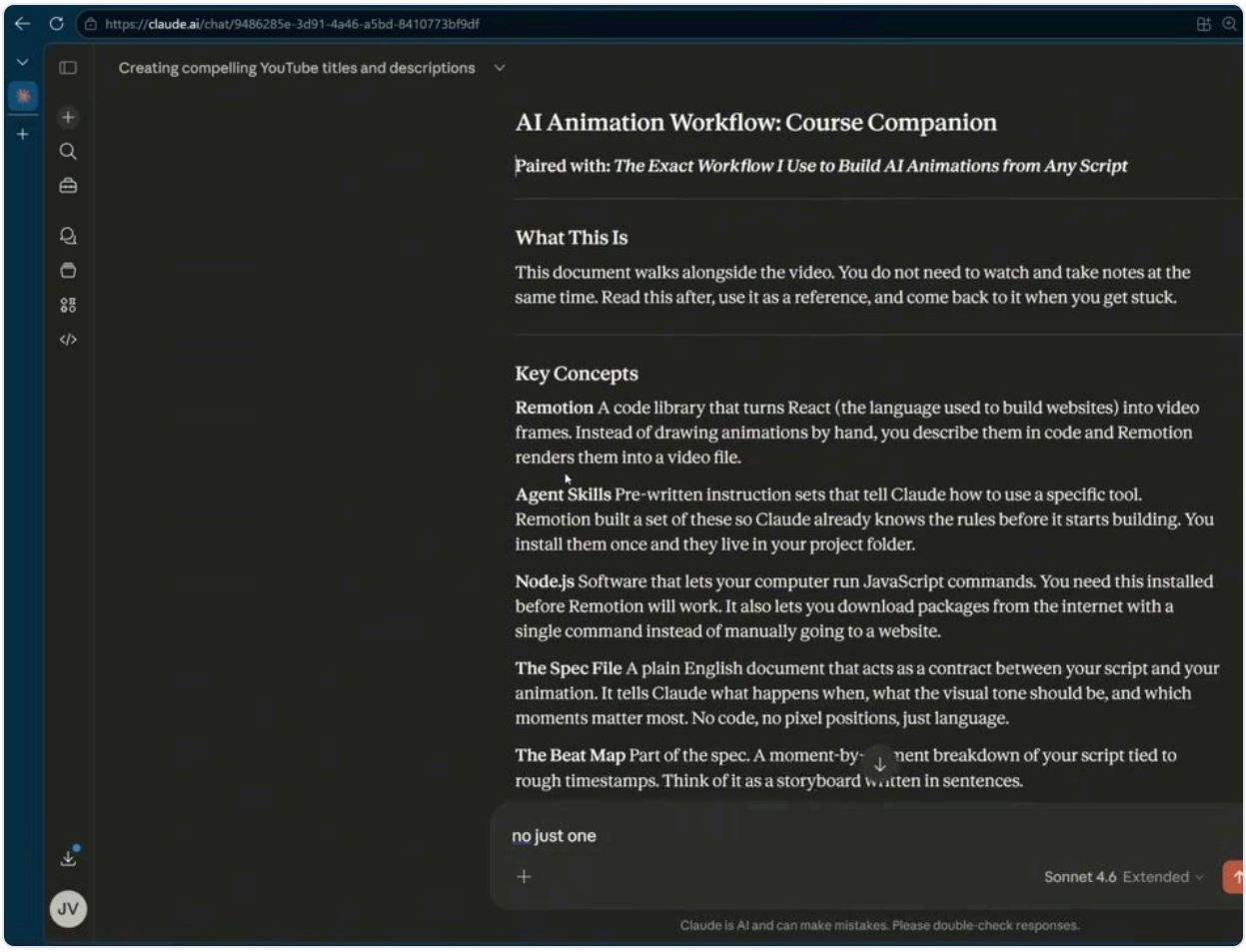


The blueprint as a folder tree: separate community, production, and writing room workspaces, plus the shared files at the top.

Plain text and a folder view

The files in these workspaces are **Markdown** files, which are just text files with some lightweight formatting. Dashes make bullets, pound signs make headings, and asterisks make text bold. It reads like a plain document, so even though it can look technical, it is all natural language.

Markdown was created in 2004 by John Gruber, and the whole idea was to write something that is readable as plain text but can also render into a nicely formatted document. Your AI already writes in Markdown: those bold words and headings you see in a chat reply are Markdown being rendered to look that way.



The same Markdown text rendered as a clean, formatted document inside an AI chat.

The walkthrough uses a code editor called VS Code to show the files, because it lets you open a whole folder and jump between files without double clicking each one. You do not need it. You can do the same thing inside Claude, or even open the files in Notepad. Nothing breaks when you edit them, because it is just English.

The three layer system

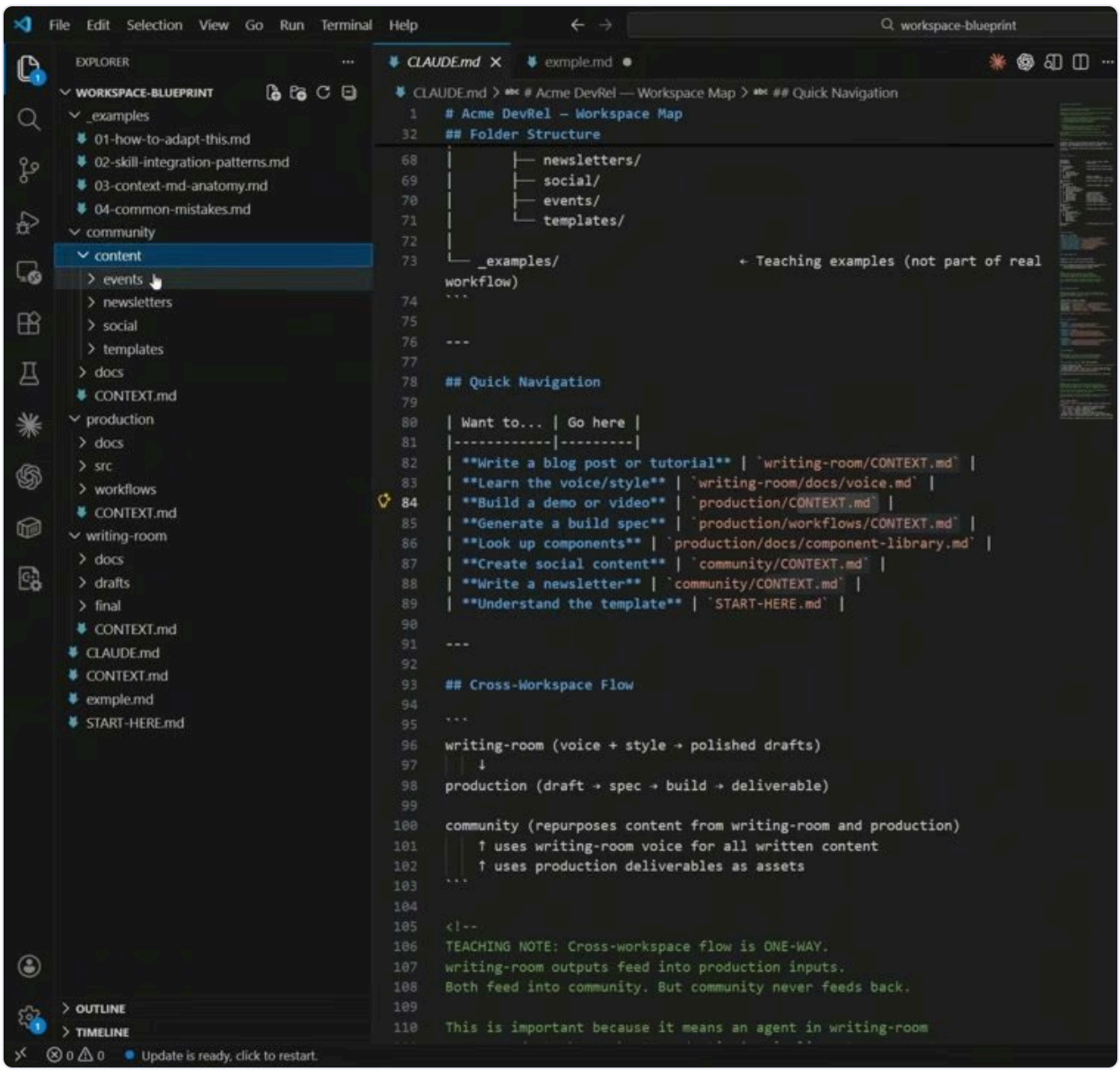
The folder runs on three layers, and each one has a job. The walkthrough uses a file named **CLAUDE.md** as the first layer. This is a file the AI reads every single time it works in the folder, so it always knows the basics without you pasting them in.

1. **Layer one is the map.** This loads automatically. It holds what the AI always needs: the folder structure, naming conventions, and where files go. Think of it as the floor plan on the wall of every room.
2. **Layer two is the rooms.** These are the per workspace context files the map points you to. Want to write a blog post? The map sends you to the writing room's context file. Want to build a demo? It sends you to production's.
3. **Layer three is the workspace itself.** These are the actual folders and files: where your drafts, events, newsletters, and finished work live.

The map loads when you start any task. A workspace file loads only when you are in that workspace. So when you do production work, the AI only reads production files, and when you are in the writing room, it only reads writing room files. Most people only build one of these layers, maybe two. Using all three is what stops the AI from doing too much at once while still letting you edit every part of the process.

The routing table

The single most important pattern in the whole system is a simple table inside the map. It is written in plain English: for a given task, read these files, skip those files, and you might need these skills. That is it. No code, just file and folder names describing what you want.



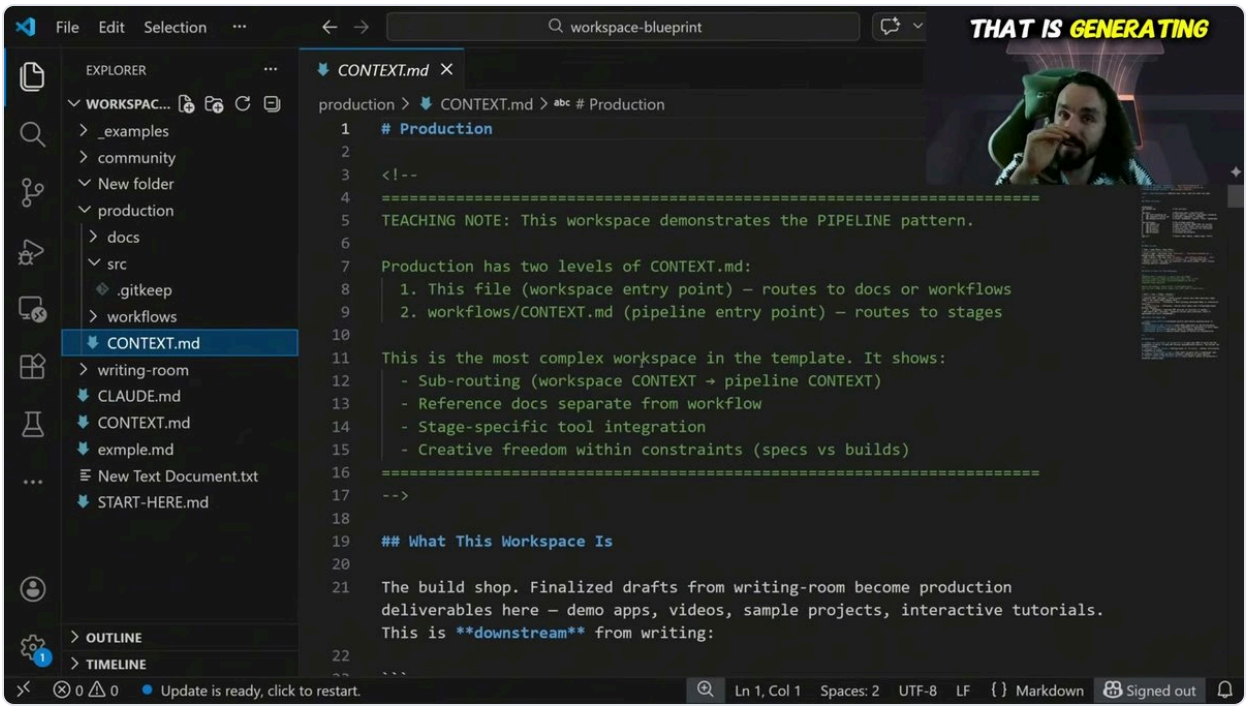
The routing table in the map file: a 'Want to' column on the left and the exact file to open on the right.

> **WARNING**
>
> Without this table, the AI either reads everything and burns the context window, guesses wrong about what matters, or produces work you cannot easily edit. The table removes all three problems at once.

This is not a new trick. Routing a request to the right handler is something software has done for decades. The only difference now is that the routing is written in plain English instead of code, so you can read it and swap it around just by looking at it.

Context files route deeper

Inside a workspace, its own context file does the next level of routing. Production is the most involved example. Its context file routes to a pipeline with four stages: a brief, a spec, a build, and the output. A script idea becomes a brief, the brief generates a spec, the spec drives the build, and the build produces the finished output.



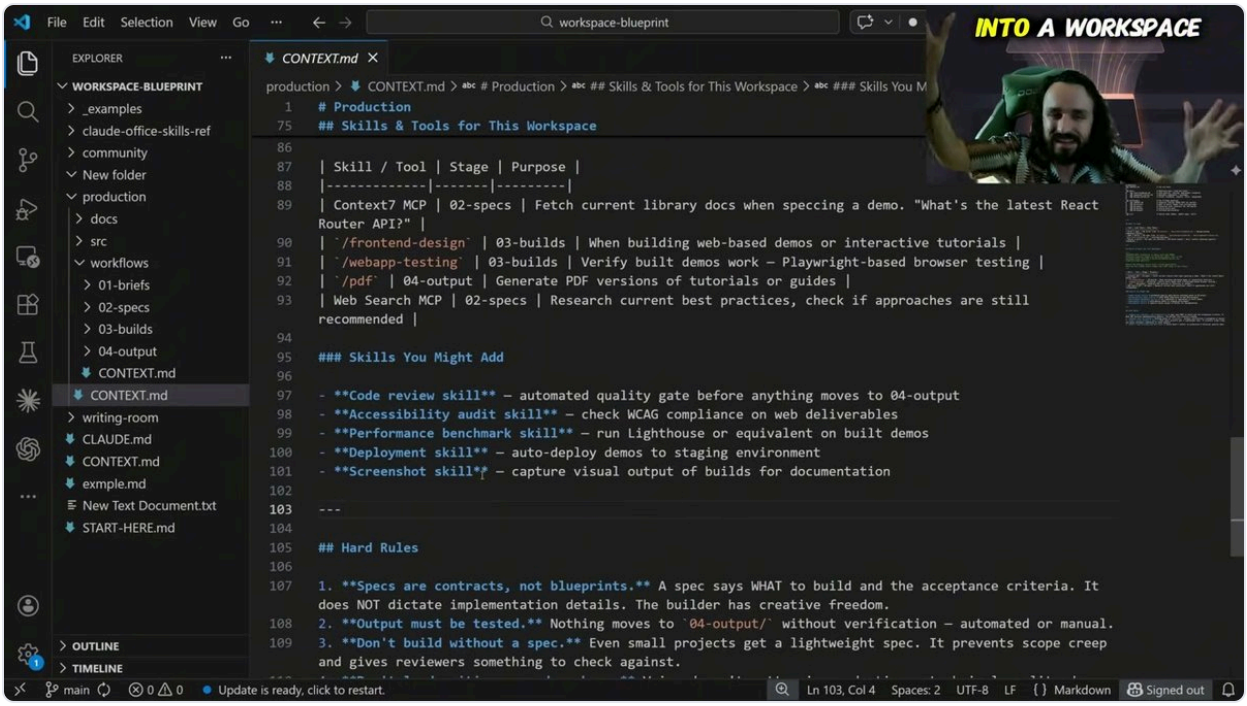
A workspace context file that routes deeper: the production workspace has its own entry point that points to stages and reference docs.

This is why one context file can do so much. It can also pull in reference docs only when a stage needs them. The brief stage might load your technical standards, and the spec stage might load your design system and component library. You point each stage at exactly what it needs and nothing more.

Wiring in skills and tools

A **skill** is a process someone else figured out and packaged up, as a set of files that tell the AI how to do a specific task. You can download them, like a humanizer skill or a document co-authoring skill from a public repository. On their own they are just available. The difference here is that you put skills inside your context files, wired into the thought process instead of floating loose.

The same context files can also call connected tools through something called **MCP**, which is just a standard way for the AI to talk to other apps and services without custom setup for each one. So a workspace might reach for a design skill, a testing skill, or a tool that looks up current information, but only at the point in the work where it makes sense.

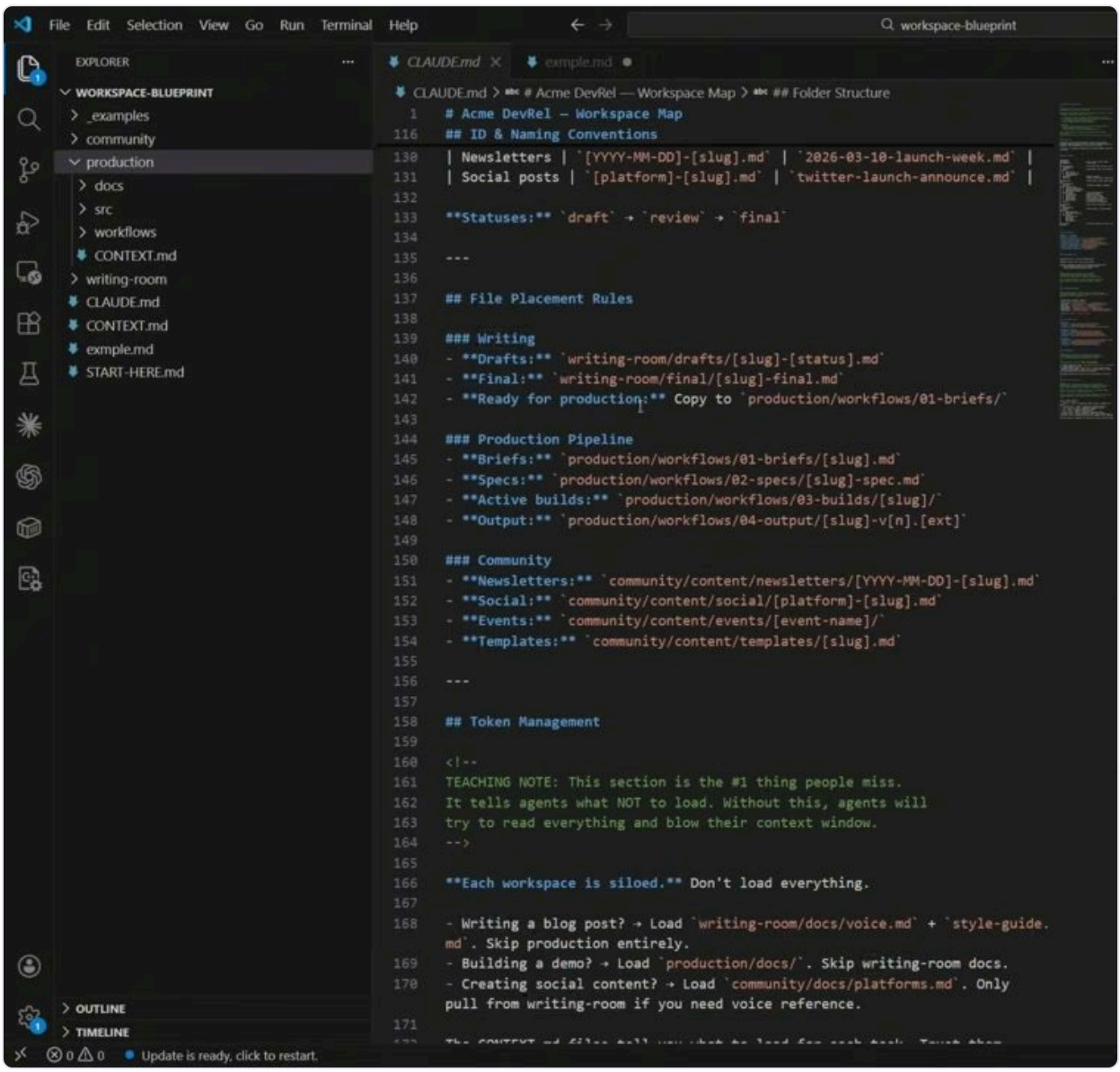


Skills and tools listed in a workspace context file, each with a note on when to use it and what it is for.

> **TIP**
>
> You could wire in a hundred skills, but the better move is to place each one only in the workspace and stage where it is needed, rather than loading them all the time. The whole idea is plug and play, on demand.

Naming instead of a database

One quiet trick replaces a lot of complicated machinery. In the map file, you add naming conventions: a rule for how each kind of file should be named and where it lives. A blog draft might be named for its slug and status, and a newsletter might lead with the year and date. Because the rule is written down, the AI knows how to name new files and where to put them.



Naming and file placement rules in the map: each kind of file has a clear pattern and a home folder.

That naming is enough for the AI to find things on its own. You can ask it to pull a specific draft and build a spec from it, and it knows where that file should be, reads the related docs, and gets to work. There is no database, no query language, and no extra code involved. The folder is doing the job that people often reach for heavier tools to solve.

Make it yours

The template ships with a made up example: fake blog posts, demos, and processes. The work is to swap that for your own. A content creator might turn the writing room into a script lab,

production into an edit bay, and community into a distribution hub. A developer or freelancer might swap writing and production for intake, build, and delivery. You change the names, the rules, and the tone to match your audience.

> **TIP**
> The workspaces change, but the three layer routing system stays the same: a map sends you to a workspace, the workspace sends you to lower level context, and that points to the right skills. With one subscription you can spin up many of these folder workspaces, each one shaped to a different job.

This is not a prompt trick or heavy infrastructure. It is folders and Markdown files arranged with the discipline of good software design. Once it is in place, every conversation after that knows where it is, what to load, which tools to use, and where to put the finished work.

What you have now

You now understand the whole pattern. Separate work into focused workspaces so the AI never reads more than it needs. Use a map file that loads every time, per workspace context files that route deeper, and the actual folders underneath. Drive it all with a plain English routing table, wire skills in where they belong, and lean on naming conventions instead of a database.

> **NOTE**
> Start small. Build one workspace with a map and a routing table, get comfortable with it, then add the next. The habits carry straight over as you grow the system.

Tools and links

- Claude and Claude Code: claude.com
- Visual Studio Code (the editor used to view the folders): code.visualstudio.com
- Markdown (created by John Gruber): daringfireball.net/projects/markdown
- MCP, the model context protocol for connecting tools: modelcontextprotocol.io

Use This Folder System Instead.' (public video). Attributed to the original presenter.